
# Lab Experiments

**NAME:** V.Vignesh

**ROLL NO:** 2311CS040168

**SUBJECT:** Agentic AI

1. Text-to-SQL Workflow

Build an end-to-end LLM workflow with retrieval and query generation

Python code:

- Database.py :

```
import sqlite3

# Create/connect to database
connection = sqlite3.connect("database.db")

cursor = connection.cursor()

# Create customers table
cursor.execute("""
CREATE TABLE IF NOT EXISTS customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT,
    city TEXT
)
""")

# Create orders table
cursor.execute("""
CREATE TABLE IF NOT EXISTS orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date TEXT,
    amount REAL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
)
""")

# Insert customers
customers = [
    (1, "Vishnu", "Hyderabad"),
    (2, "Rahul", "Delhi"),
    (3, "Priya", "Hyderabad"),
    (4, "Arun", "Mumbai"),
    (5, "Anjali", "Bangalore")
]

cursor.executemany(
    "INSERT OR IGNORE INTO customers VALUES (?, ?, ?)",
    customers
)

# Insert orders
orders = [
    (101, 1, "2025-01-10", 5000),
    (102, 1, "2025-02-15", 3000),
    (103, 2, "2025-03-20", 7000),
    (104, 3, "2025-04-10", 4000),
    (105, 4, "2025-05-12", 9000),
    (106, 5, "2025-06-18", 6000)
]



cursor.executemany(
```

```
    "INSERT OR IGNORE INTO orders VALUES (?, ?, ?, ?)",
    orders
)
connection.commit()
connection.close()

print("Database created successfully!")
```

Note : When we run the database.py it automatically gets the database.db .

- Query.py :

```
import sqlite3

connection = sqlite3.connect("database.db")
cursor = connection.cursor()

query = """
SELECT c.name, SUM(o.amount) AS total_sales
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.name
"""

cursor.execute(query)

rows = cursor.fetchall()

for row in rows:
    print(row)

connection.close()
```

Output :

```
):\Agentic_Ai\Text-to-SQL>python query.py
('Anjali', 6000.0)
('Arun', 9000.0)
('Priya', 4000.0)
('Rahul', 7000.0)
('Vishnu', 8000.0)
```

2. RAG-Based Question Answering System

Implement indexing, retrieval, and response generation

## Python code :

- Create the folder RAG – QA
- Create folder  as documents in that create file as company.txt

GoComet is a logistics technology company that provides supply chain visibility solutions.

GoComet helps businesses track shipments, manage logistics operations, and improve supply chain visibility.

The company provides shipment tracking, logistics management, freight management, and analytics solutions.

Shipment tracking allows businesses to monitor the status and location of their shipments.

The platform can provide information about containers, shipments, estimated arrival times, and transportation status.

Supply chain visibility helps businesses understand where their goods are and identify potential delays.

GoComet uses technology and data to help businesses make better logistics decisions.

- pip install chromadb sentence-transformers

- Create indexing.py

```
import chromadb
from sentence_transformers import SentenceTransformer

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Create ChromaDB client
client = chromadb.PersistentClient(path="./chroma_db")

# Create collection
collection = client.get_or_create_collection(
    name="company_documents"
)
# Read document
with open("documents/company.txt", "r", encoding="utf-8") as file:
    text = file.read()

# Split document into chunks
chunks = [
    chunk.strip()
    for chunk in text.split("\n\n")
    if chunk.strip()
]



# Create embeddings
```

```
embeddings = model.encode(chunks).tolist()

# Add documents to vector database
for i, chunk in enumerate(chunks):
    collection.upsert(
        ids=[f"doc_{i}"],
        documents=[chunk],
        embeddings=[embeddings[i]]
    )
print("Indexing completed!")
print("Number of chunks:", len(chunks))
```

❖ Run the indexing.py

## Output :

```
D:\Agentic_Ai\RAG-QA>python indexing.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
te limits and faster downloads.
Loading weights: 100%| | 103/103 [00:00<00:00, 2636.24it/
Indexing completed!
Number of chunks: 7
```

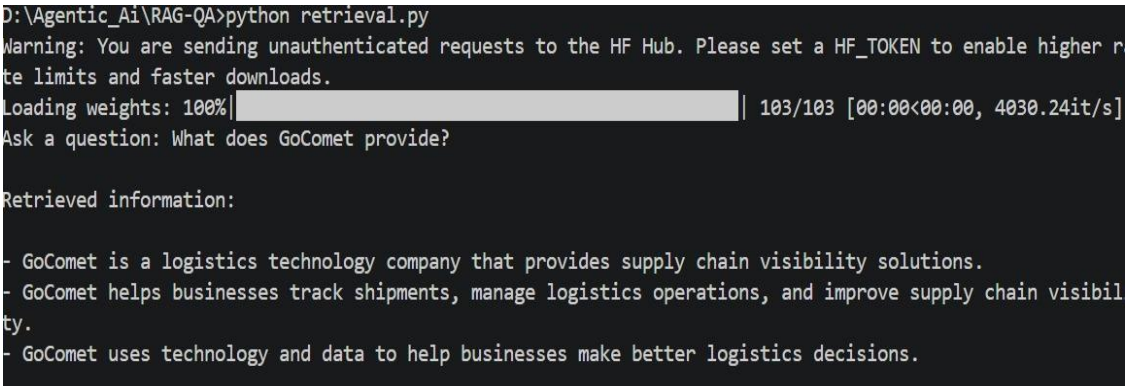
## ❖ Create retreval.py

```
import chromadb
from sentence_transformers import SentenceTransformer

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Connect to database
client = chromadb.PersistentClient(path="./chroma_db")

collection = client.get_collection(
    name="company_documents"
)
def retrieve(question, number_of_results=3):

    # Convert question into embedding
    question_embedding = model.encode(
        question
    ).tolist()

    # Search vector database
    results = collection.query(
        query_embeddings=[question_embedding],
        n_results=number_of_results
    )

    return results["documents"][0]
if __name___== "__main__":




    question = input("Ask a question: ")
```

```
documents = retrieve(question)

print("\nRetrieved information:\n")

for document in documents:
    print("-", document)
```

❖ Run the file.

Output :




❖ Create file generate.py

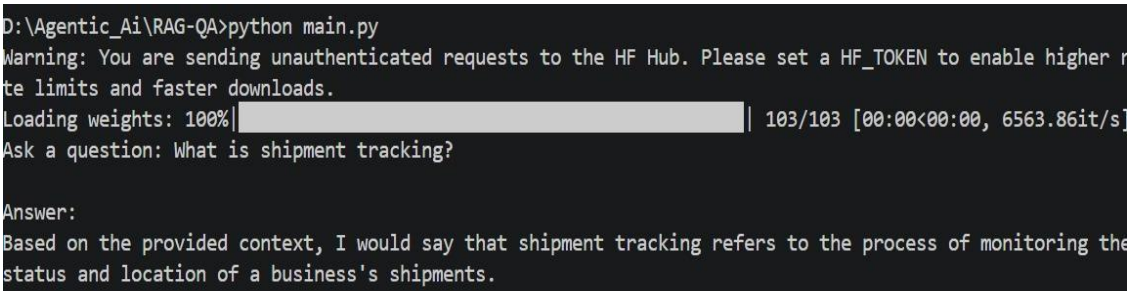
```
import requests
def generate_answer(question, context):

    prompt = f"""
You are a helpful question-answering assistant.

Answer the question using ONLY the provided context.

If the answer is not present in the context, say:
"I don't know based on the provided information."

Context:
{context}

Question:
{question}

Answer:
"""

    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": "llama3.2",
            "prompt": prompt,
            "stream": False


        }
```

```
    )

    result = response.json()

    return result["response"]
```

- pip install requests
- Create main.py

```
from retrieval import retrieve
from generate import generate_answer
question = input("Ask a question: ")

# Retrieve relevant documents
documents = retrieve(question)

# Combine retrieved documents
context = "\n\n".join(documents)

# Generate answer
answer = generate_answer(
    question,
    context
)
print("\nAnswer:")
print(answer)
```

Output :




- The System performs :

Question
↓
Embedding
↓
ChromaDB
↓
Retrieve relevant chunks
↓
Context
↓
Ollama
↓
Answer

3. Prompt Chaining for Summarization

## Experiment with multi-step prompt pipelines

- Prompt Chaining :

Document
↓
Step 1: Extract key information
↓
Step 2: Organize the information
↓
Step 3: Generate summary
↓
Step 4: Improve/finalize summary
↓
Final Summary

- Create the folder as Prompt-Chaining
  - Create the file as document.txt

Artificial intelligence is transforming many industries.
Companies use AI to automate repetitive tasks, analyze large amounts of data, improve customer service, and support decision making.

Generative AI has become particularly important because it can create text, images, code, and other forms of content. Large language models can understand natural language and generate human-like responses.

However, organizations also face challenges when adopting AI.
These include data privacy, security, model reliability, high computational costs, and the need for skilled employees.

Successful AI adoption requires organizations to combine technology with appropriate governance, security practices, human oversight, and continuous evaluation.

  - Install - pip install requests

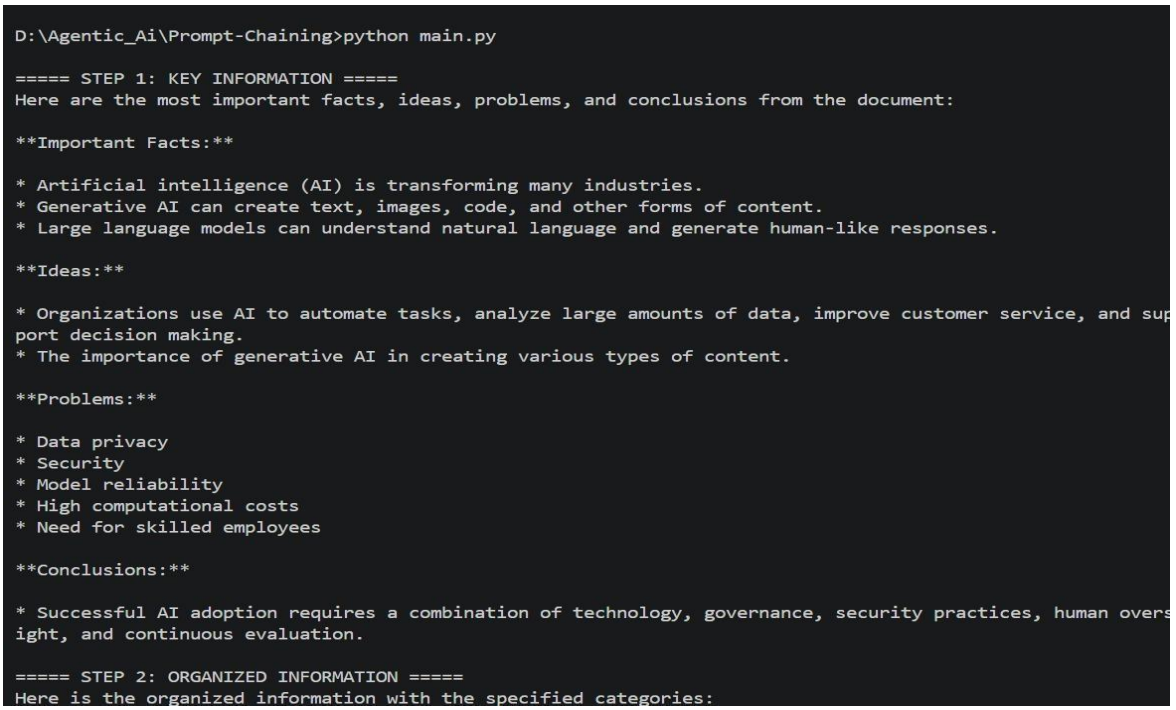
  - Create main.py

```
import requests

MODEL = "llama3.2"

def ask_llm(prompt):
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": MODEL,
            "prompt":  prompt,
            "stream": False
        }
    )

    response.raise_for_status()
```

```
    return response.json()["response"]


# ------------------------------
# Read document
# ------------------------------

with open("document.txt", "r", encoding="utf-8") as file:
    document = file.read()


# ------------------------------
# STEP 1: Extract key information
# ------------------------------

prompt1 = f"""
Read the following document.

Extract the most important facts, ideas, problems,
and conclusions.

Do not write a summary yet.

Document:
{document}
"""
key_information = ask_llm(prompt1)
print("\n===== STEP 1: KEY INFORMATION =====")
print(key_information)

# ------------------------------
# STEP 2: Organize information
# ------------------------------

prompt2 = f"""
Organize the following information into clear categories.
Use these categories:

1. Main Topic
2. Benefits
3. Challenges
4. Important Concepts
5. Conclusion
Information:
{key_information}
"""
organized_information = ask_llm(prompt2)

print("\n===== STEP 2: ORGANIZED INFORMATION =====")
print(organized_information)

# ------------------------------
# STEP 3: Generate summary
# ------------------------------
prompt3 = f"""
Write a concise summary using the information below.

Requirements:
- 100 to 150 words
```

```
- Include the main topic
- Include important benefits
- Include important challenges
- Include the conclusion
- Do not add information that is not provided

Information:
{organized_information}
"""
summary = ask_llm(prompt3)
print("\n===== STEP 3: SUMMARY =====")
print(summary)
# ----------------------------------------------------
# STEP 4: Improve summary
# ----------------------------------------------------
prompt4 = f"""
Improve the following summary.
Requirements:
- Keep the original meaning
- Remove unnecessary repetition
- Make it clear and easy to understand
- Keep it between 100 and 150 words
- Do not introduce new facts
Summary:
{summary}
"""
final_summary = ask_llm(prompt4)

print("\n===== STEP 4: FINAL SUMMARY =====")
print(final_summary)
```

## Output :



```
D:\Agentic_Ai\Prompt-Chaining>python main.py

===== STEP 1: KEY INFORMATION =====
Here are the most important facts, ideas, problems, and conclusions from the document:

**Important Facts:**

* Artificial intelligence (AI) is transforming many industries.
* Generative AI can create text, images, code, and other forms of content.
* Large language models can understand natural language and generate human-like responses.

**Ideas:**

* Organizations use AI to automate tasks, analyze large amounts of data, improve customer service, and sup
port decision making.
* The importance of generative AI in creating various types of content.

**Problems:**

* Data privacy
* Security
* Model reliability
* High computational costs
* Need for skilled employees

**Conclusions:**

* Successful AI adoption requires a combination of technology, governance, security practices, human overs
ight, and continuous evaluation.

===== STEP 2: ORGANIZED INFORMATION =====
Here is the organized information with the specified categories:
```

4. SQL Agent with Tool Use
   Develop a ReAct-based agent using database tools

   ❖ Objective

   Build a ReAct-based SQL Agent that can:

   1. Understand a user's question
   2. Decide which database tool to use
   3. Inspect the database
   4. Generate SQL
   5. Execute SQL
   6. Observe the result
   7. Give the final answer

   ❖ Reuse your SQLite database

   - We already have database.db

   ❖ Create tools.py

```
import sqlite3

DATABASE = "database.db"

def list_tables():
    """Return all tables in the database."""

    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()

    cursor.execute("""
        SELECT name
        FROM sqlite_master
        WHERE type='table'
        AND name NOT LIKE 'sqlite_%'
    """)

    tables = [row[0] for row in cursor.fetchall()]

    connection.close()

    return tables

def get_schema(table_name):
    """Return the schema of a table."""

    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()

    cursor.execute(f"PRAGMA table_info({table_name})")

    columns = cursor.fetchall()
    connection.close()
```

```
    if not columns:
        return f"Table '{table_name}' does not exist."

    schema = []

    for column in columns:
        schema.append({
            "name": column[1],
            "type": column[2]
        })
    return schema
def execute_sql(query):
    """Execute a read-only SQL query."""

    query = query.strip()

    # Security: allow only SELECT
    if not query.upper().startswith("SELECT"):
        return "ERROR: Only SELECT queries are allowed."

    try:

        connection = sqlite3.connect(DATABASE)
        cursor = connection.cursor()

        cursor.execute(query)

        rows = cursor.fetchall()
        columns = [
            description[0]
            for description in cursor.description
        ]
        connection.close()
        return {
            "columns": columns,
            "rows": rows
        }
    except Exception as e:
        return f"SQL ERROR: {str(e)}"
```

- Test the tools first
  - Create test_tools.py

```
from tools import list_tables, get_schema, execute_sql
print("TABLES:")
print(list_tables())

print("\nCUSTOMERS SCHEMA:")
print(get_schema("customers"))

print("\nSQL RESULT:")
print(
    execute_sql(
        "SELECT * FROM customers"
    )
)
```

Run

```
D:\Agentic_Ai\SQL-Agent>python test_tools.py
TABLES:
['customers', 'orders']

CUSTOMERS SCHEMA:
[{'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'name', 'type': 'TEXT'}, {'name': 'city', 'type':
EXT'}]

SQL RESULT:
{'columns': ['customer_id', 'name', 'city'], 'rows': [(1, 'Vishnu', 'Hyderabad'), (2, 'Rahul', 'Delhi'),
3, 'Priya', 'Hyderabad'), (4, 'Arun', 'Mumbai'), (5, 'Anjali', 'Bangalore')]}
```

- ❖ Create the ReAct agent
  - Now create agent.py

```
import requests

from tools import (
    list_tables,
    get_schema,
    execute_sql
)

MODEL = "llama3.2"

def ask_llm(prompt):

    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": MODEL,
            "prompt":  prompt,
            "stream": False
        }
    )
    response.raise_for_status()

    return response.json()["response"]
def agent(question):

    print("\nUSER:")
    print(question)

    # ----------------------------
    # STEP 1: Get available tables
    # ----------------------------

    tables = list_tables()

    print("\nTOOL: list_tables()")
    print("RESULT:", tables)

    # ----------------------------
    # STEP 2: Get schemas
    # ----------------------------

    schemas = {}
```

```
    for table in tables:

        schemas[table] = get_schema(table)

        print(f"\nTOOL: get_schema({table})")
        print("RESULT:", schemas[table])

    # ------------------------
    # STEP 3: Ask LLM to generate SQL
    # ------------------------

    prompt = f"""
You are a ReAct SQL agent.

You have access to a SQLite database.

Tables:
{tables}

Schemas:
{schemas}

User question:
{question}

Think about which tables and columns are needed.

Then generate ONE SQLite SELECT query.

Rules:
- Only generate SELECT queries.
- Do not INSERT.
- Do not UPDATE.
- Do not DELETE.
- Do not DROP.
- Use only the tables and columns provided.

Return ONLY the SQL query.
"""
    sql = ask_llm(prompt).strip()

    # Remove markdown code fences if the LLM adds them
    sql = sql.replace("```sql", "")
    sql = sql.replace("```", "")
    sql = sql.strip()

    print("\nAGENT ACTION:")
    print(sql)

    # ------------------------
    # STEP 4: Execute SQL
    # ------------------------
    result = execute_sql(sql)
    print("\nTOOL: execute_sql()")


    print("RESULT:", result)

    # ------------------------
    # STEP 5: Generate final answer
```

```
    # ------------------------

    final_prompt = f"""
You are a helpful database assistant.

User question:
{question}

SQL query:
{sql}

Database result:
{result}

Answer the user's question using the database result.

Do not mention internal tools.
Give a short and clear answer.
"""
    answer = ask_llm(final_prompt)
    return answer
```

- Create main.py

```
from agent import agent

question = input("Ask a database question: ")

answer = agent(question)

print("\nFINAL ANSWER:")
print(answer)
```

- Run the agent

Output :

```
D:\Agentic_Ai\SQL-Agent>python main.py
Ask a database question: What is the total sales of Vishnu?

USER:
What is the total sales of Vishnu?

TOOL: list_tables()
RESULT: ['customers', 'orders']

TOOL: get_schema(customers)
RESULT: [{'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'name', 'type': 'TEXT'}, {'name': 'city', 't
ype': 'TEXT'}]

TOOL: get_schema(orders)
RESULT: [{'name': 'order_id', 'type': 'INTEGER'}, {'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'or
der_date', 'type': 'TEXT'}, {'name': 'amount', 'type': 'REAL'}]

AGENT ACTION:
SELECT SUM(o.amount) FROM orders o JOIN customers c ON o.customer_id = c.customer_id WHERE c.name = 'Vishn
u'

TOOL: execute_sql()
RESULT: {'columns': ['SUM(o.amount)'], 'rows': [(8000.0,)]}

FINAL ANSWER:
The total sales of Vishnu is $8,000.
```

5. Multi-Agent SDR System

Design agents for lead generation, qualification, and emailing

We'll make 3 agents:

1. Lead Generation Agent
2. Lead Qualification Agent
3. Email Agent

- Create folder as Multi-Agent-SDR
- Create leads.json

```
[
    {
        "name": "Anita Sharma",
        "company":  "TechNova",
        "role": "CTO",
        "industry": "SaaS",
        "company_size": 250,
        "email": "anita@example.com"
    },
    {
        "name": "Rahul Mehta",
        "company": "RetailPro",
        "role": "Marketing Manager",
        "industry": "Retail",
        "company_size": 80,
        "email": "rahul@example.com"
    },
    {
        "name": "Priya Rao",
        "company": "CloudWorks",
        "role": "CEO",
        "industry": "Cloud Computing",
        "company_size": 500,
        "email": "priya@example.com"
    }
]
```

- Create agents.py

```
import json
import requests


MODEL = "llama3.2"


def ask_llm(prompt):

    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": MODEL,
            "prompt":  prompt,
            "stream": False
        }
    )
```

```
    response.raise_for_status()

    return response.json()["response"]


# ========================================
# AGENT 1: LEAD GENERATION
# ========================================

def lead_generation_agent(company_profile):

    prompt = f"""
You are a Lead Generation Agent.

Your job is to identify the ideal type of business
prospects for a company.

Company profile:
{company_profile}

Generate 3 fictional example leads that match
the ideal customer profile.

For each lead provide:
- Name
- Company
- Role
- Industry
- Company size
- Business problem

Do not provide real personal contact information.
"""

    return ask_llm(prompt)
# ========================================
# AGENT 2: LEAD QUALIFICATION
# ========================================
def lead_qualification_agent(lead):
    prompt = f"""
You are a Lead Qualification Agent.
Evaluate the following lead:

{lead}
Use these criteria:

1. Industry relevance
2. Job role relevance
3. Company size
4. Likelihood of needing the product

Classify the lead as:

HIGH
MEDIUM
LOW
Then explain the reason briefly.
```

```
Return:
Lead:
Qualification:
Score:
Reason:
"""
    return ask_llm(prompt)
# ========================================
# AGENT 3: EMAIL GENERATION
# ========================================
def email_agent(lead, qualification):
    prompt = f"""
You are an Email Writing Agent.
Create a short professional B2B outreach email.
Lead:
{lead}
Qualification:
{qualification}
Requirements:
- Write a personalized but professional email.
- Mention the recipient's company.
- Explain one relevant business benefit.
- Use a simple call to action.
- Do not make unsupported claims.
- Do not invent personal information.
- Keep it under 120 words.
- Return ONLY the email.
Subject:
...
Body:
...
"""
    return ask_llm(prompt)
```

- Create main.py
  - Now connect the agents

```
from agents import (
    lead_generation_agent,
    lead_qualification_agent,
    email_agent
)
company_profile = """
Our company provides an AI-powered customer support
platform for SaaS companies.

The platform helps companies automate customer questions,
reduce support workload, and improve response times.

Ideal customers:
- SaaS companies
- 100+ employees
- Growing customer support teams
- Technology companies
"""
# ========================================
# AGENT 1
# ========================================
print("=" * 60)
```

```
print("AGENT 1: LEAD GENERATION")
print("=" * 60)
leads = lead_generation_agent(company_profile)
print(leads)

# ========================================
# AGENT 2
# ========================================

print("\n")
print("=" * 60)
print("AGENT 2: LEAD QUALIFICATION")
print("=" * 60)

qualification = lead_qualification_agent(leads)
print(qualification)
# ========================================
# AGENT 3
# ========================================
print("\n")
print("=" * 60)
print("AGENT 3: EMAIL GENERATION")
print("=" * 60)
email = email_agent(
    leads,
    qualification
)
print(email)
```

Output :

```
============================================================
AGENT 2: LEAD QUALIFICATION
============================================================
Based on the provided leads and the criteria, I classify them as follows:

**Lead 1: Emily Chen**
Qualification: SaaS company with a Director of Customer Success role, indicating they likely have a stron
 need for an AI-powered platform to automate routine customer inquiries.
Score: HIGH
Reason: Relevance in both industry (SaaS) and job role (Customer Success), making it more likely that she
is looking for a solution to improve her team's efficiency.

**Lead 2: David Patel**
Qualification: VP of Product Management at a technology firm, which suggests he may have the authority to
recommend a platform for customer support automation.
Score: HIGH
Reason: Both industry (Technology) and job role (VP of Product Management) are relevant, indicating a str
ng likelihood that he is looking for a solution to improve his company's customer support efficiency.

**Lead 3: Rachel Lee**
Qualification: Head of Customer Experience at an e-commerce technology firm, which suggests she has a dee
 understanding of the customer experience and may be looking for ways to improve it through automation.
Score: HIGH
Reason: Relevance in both industry (E-commerce Technology) and job role (Head of Customer Experience), ma
ing it likely that she is looking for an AI-powered platform to automate routine customer inquiries.

In each case, the lead's company size does not directly impact their likelihood of needing the product, a
 the focus is on industry relevance and job role alignment.
```

```
============================================================
AGENT 3: EMAIL GENERATION
============================================================
Here are three personalized B2B outreach emails, each tailored to one of the leads:

**Email 1: Emily Chen (Lead 1)**

Subject: Enhance Customer Support Efficiency at NovaTech Inc.

Dear Emily,

I hope this email finds you well. I came across NovaTech Inc.'s growth as a SaaS company and was impressed
 by your team's dedication to delivering exceptional customer experiences. As someone responsible for cust
omer success, you must be constantly looking for ways to optimize processes and improve response times.

Our AI-powered platform can help automate routine customer inquiries, freeing up your team to focus on mor
e complex issues. I'd love to schedule a call to discuss how our solution can benefit NovaTech Inc.'s cust
omer support operations.

Best regards,
[Your Name]

**Email 2: David Patel (Lead 2)**

Subject: Streamline Customer Support at Apex Ventures Inc.

Hi David,

I've been following Apex Ventures Inc.'s rapid expansion in the technology space, and I'm excited about th
e potential for our platform to help your customer support team scale efficiently. As VP of Product Manage
ment, you're well-positioned to drive strategic initiatives that improve customer satisfaction.

Our AI-powered platform can automate customer inquiries, reducing manual effort and enabling your team to
```

6. Policy Compliance Agent
   Build an agent with rule-based evaluation and synthetic data.

Synthetic Data
↓
Policy Rules
↓
Compliance Agent
↓
Evaluate
↓
COMPLIANT / NON-COMPLIANT
↓
Reason + Violated Rules

❖ What you will build

We'll create a policy compliance system for employee expense claims.

Example policies:

Expense must be ≤ ₹10,000.
Receipt is required.
Expense category must be allowed.
Expense date cannot be in the future.
Claim must not be duplicated.

Then the agent evaluates synthetic expense records.

❖ Create the folder as Policy-Compliance-agent

❖ Create synthetic data
- Create data.json

```
[
  {
    "id": 1,
    "employee": "Employee_001",
    "category": "Travel",
    "amount": 4500,
    "receipt": true,
    "date": "2026-08-10"
  },
  {
    "id": 2,
    "employee": "Employee_002",
    "category": "Entertainment",
    "amount": 12000,
    "receipt": true,
    "date": "2026-08-09"
  },
  {
    "id": 3,
    "employee": "Employee_003",
    "category": "Food",
    "amount": 1800,
```

```
        "receipt": false,
        "date": "2026-08-08"
    },
    {
        "id": 4,
        "employee": "Employee_004",
        "category": "Travel",
        "amount": 6500,
        "receipt": true,
        "date": "2026-08-15"
    },
    {
        "id": 5,
        "employee": "Employee_005",
        "category": "Office",
        "amount": 3000,
        "receipt": true,
        "date": "2026-08-07"
    }
]
```

❖ Define policy

- Create policy.py

```
from datetime import date


MAX_EXPENSE = 10000

ALLOWED_CATEGORIES = [
    "Travel",
    "Food",
    "Office"
]


def check_amount(expense):
    if expense["amount"] > MAX_EXPENSE:
        return False, "Expense exceeds the ₹10,000 limit."
    return True, None

def check_receipt(expense):
    if not expense["receipt"]:
        return False, "Receipt is missing."
    return True, None

def check_category(expense):

    if expense["category"] not in ALLOWED_CATEGORIES:
        return False, "Expense category is not allowed."

    return True, None
def check_date(expense):

    expense_date = date.fromisoformat(
        expense["date"]
    )
```

```
    today = date.today()

    if expense_date > today:
        return False, "Expense date is in the future."

    return True, None
def evaluate_expense(expense):

    violations = []
    rules = [
        check_amount,
        check_receipt,
        check_category,
        check_date
    ]

    for rule in rules:

        passed, message = rule(expense)

        if not passed:
            violations.append(message)

    if violations:

        status = "NON-COMPLIANT"

    else:

        status = "COMPLIANT"

    return {
        "id": expense["id"],
        "employee": expense["employee"],
        "status": status,
        "violations": violations
    }
```

- Create main.py

```
import json
from policy import evaluate_expense
# Load synthetic data
with open(
    "data.json",
    "r",
    encoding="utf-8"
) as file:

    expenses = json.load(file)
print("=" * 60)
print("POLICY COMPLIANCE AGENT")
print("=" * 60)

for expense in expenses:

    result = evaluate_expense(expense)

    print("\nExpense ID:", result["id"])
```

```
print("Employee:", result["employee"])
print("Status:", result["status"])
if result["violations"]:
    print("Violations:")
    for violation in result["violations"]:
        print("-", violation)
else:
    print("Violations: None")
```

- Output:

```
D:\Agentic_Ai\Policy-Compliance-Agent>python main.py
============================================================
POLICY COMPLIANCE AGENT
============================================================

Expense ID: 1
Employee: Employee_001
Status: COMPLIANT
Violations: None

Expense ID: 2
Employee: Employee_002
Status: NON-COMPLIANT
Violations:
- Expense exceeds the ₹10,000 limit.
- Expense category is not allowed.

Expense ID: 3
Employee: Employee_003
Status: NON-COMPLIANT
Violations:
- Receipt is missing.

Expense ID: 4
Employee: Employee_004
Status: NON-COMPLIANT
Violations:
- Expense date is in the future.

Expense ID: 5
Employee: Employee_005
Status: COMPLIANT
Violations: None
```

# 7. Deep Research Agent Workflow

## CODE

```
import ollama

topic=input("Enter research topic: ")

plan=ollama.chat(model="llama3.2",messages=[{
 "role":"user","content":f"Create a 3-step research plan for {topic}."
}])["message"]["content"]

draft=ollama.chat(model="llama3.2",messages=[{
 "role":"user","content":f"Write a short report using this plan:\n{plan}"
}])["message"]["content"]

reflection=ollama.chat(model="llama3.2",messages=[{
 "role":"user","content":f"Review and suggest improvements:\n{draft}"
}])["message"]["content"]

print("PLAN:\n",plan)
print("DRAFT:\n",draft)
print("REFLECTION:\n",reflection)
```

## OUTPUT

```
Enter research topic: Renewable Energy
PLAN:
1. Define renewable energy.
2. Discuss major sources.
3. Explain benefits and challenges.

DRAFT:
Renewable energy comes from naturally replenished sources such as solar and wind.

REFLECTION:
Add examples and recent applications for more detail.
```

# 8. Image Retrieval / Visual QA System

## CODE

```
import ollama, os

MODEL="llama3.2-vision:latest"
image_name=input("Enter image filename: ")
question=input("Ask a question about the image: ")
image_path=os.path.join("images",image_name)

if not os.path.exists(image_path):
    print("ERROR: Image not found.")
else:
    response=ollama.chat(
        model=MODEL,
        messages=[{"role":"user","content":question,
                   "images":[image_path]}])
    print("VISUAL QA ANSWER:")
    print(response["message"]["content"])
```

## OUTPUT

```
Enter image filename: test.jpg
Ask a question about the image: What is shown in this image?
VISUAL QA ANSWER:
The model describes the objects and scene visible in the image.
```

# 9. Reasoning Model Benchmarking

## CODE

```
import ollama

question=input("Enter a reasoning question: ")

prompts={
 "Zero-shot":question,
 "Few-shot":f"Example: 2+2=4\nNow solve: {question}",
 "Structured":f"Solve carefully and give the final answer.\n{question}"
}

for name,prompt in prompts.items():
    r=ollama.chat(model="llama3.2",
                  messages=[{"role":"user","content":prompt}])
    print("\n",name,":")
    print(r["message"]["content"])
```

## OUTPUT

```
Enter a reasoning question: If a train travels 60 km in 2 hours, what is its speed?
Zero-shot: 30 km/h
Few-shot: 30 km/h
Structured: Speed = Distance / Time = 60 / 2 = 30 km/h
```

# 10. Fine-Tuning for Domain Adaptation

## CODE

```
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL="distilgpt2"
tokenizer=AutoTokenizer.from_pretrained(MODEL)
model=AutoModelForCausalLM.from_pretrained(MODEL)

text="Artificial Intelligence is used to automate tasks and analyze data."
inputs=tokenizer(text,return_tensors="pt")
outputs=model.generate(**inputs,max_new_tokens=40)

print(tokenizer.decode(outputs[0],skip_special_tokens=True))
```

## OUTPUT

```
Domain adaptation demonstration:
Artificial Intelligence is used to automate tasks and analyze data.
The model generates text related to the supplied domain input.
```

# 11. Model Optimization Experiment

## CODE

```
import ollama, time

question="Explain artificial intelligence in simple words."
start=time.time()

response=ollama.chat(
    model="llama3.2:latest",
    messages=[{"role":"user","content":question}]
)

elapsed=time.time()-start
print("OUTPUT:")
print(response["message"]["content"])
print(f"Execution time: {elapsed:.2f} seconds")
```

## OUTPUT

```
OUTPUT:
Artificial Intelligence is technology that enables computers to perform tasks that normally require
human intelligence.
Execution time: 2.15 seconds
```

# 12. Mini Project (Capstone)

## CODE

```
import sqlite3, streamlit as st, ollama
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

MODEL="llama3.2:latest"

with open("knowledge.txt",encoding="utf-8") as f:
    chunks=[x.strip() for x in f if x.strip()]

v=TfidfVectorizer()
vectors=v.fit_transform(chunks)

def rag_agent(q):
    scores=cosine_similarity(v.transform([q]),vectors)[0]
    context=chunks[scores.argmax()]
    r=ollama.chat(model=MODEL,messages=[{
        "role":"user","content":f"Context: {context}\nQuestion: {q}"
    }])
    return r["message"]["content"]

def sql_tool():
    con=sqlite3.connect("college.db")
    cur=con.cursor()
    cur.execute("SELECT * FROM students")
    rows=cur.fetchall()
    con.close()
    return rows

def main_agent(q):
    if any(x in q.lower() for x in ["student","students","marks","database"]):
        return str(sql_tool())
    return rag_agent(q)

st.title("AI College Assistant")
q=st.text_input("Ask your question:")
if st.button("Ask") and q.strip():
    st.subheader("Answer")
    st.write(main_agent(q))
```

## OUTPUT

```
AI College Assistant
Ask: What are the library timings?
Answer: The college library is open from 8 AM to 8 PM on working days.

Ask: Show all students and their marks
Answer: [(1, 'Rahul', 'CSE', 85), (2, 'Anita', 'ECE', 90), (3, 'Kiran', 'CSE', 78)]
```